

Verificação, Validação e Teste de Software

Prof. Dr. Leonardo Souza Silva

Módulo 2 — Teste de Software

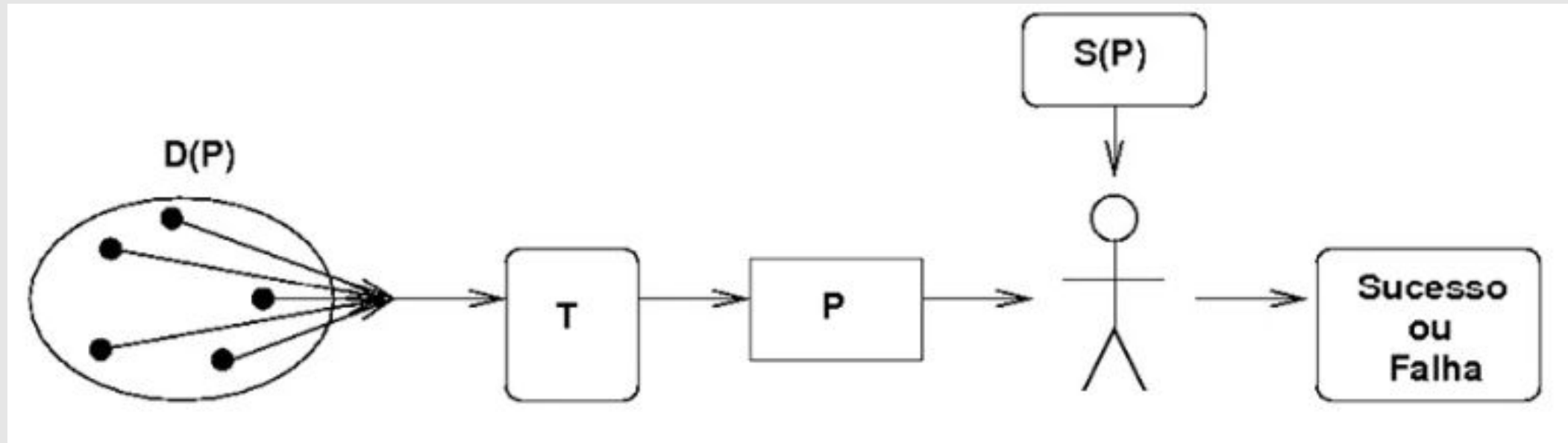
Unidade 1 - Técnicas de Teste de Software

Em nossa última aula

- Teste é o processo de executar um programa com a intenção de encontrar erros;
- Deve ser conduzido de maneira sistemática;
- Terminologia:
 - Defeito, Erro e Falha;

Mostram que o *software* tem erro, mas não provam que ele nunca falhará. Testes bem planejados podem remover em média 60% dos defeitos de um programa.

Atividade de Teste



Fonte: Delamaro (2016)

Defeitos no processo de desenvolvimento

- Gerados na comunicação e na transformação de informações;
- A maioria se encontra em partes do código raramente executadas;

Quanto mais cedo o defeito for revelado, menor o custo de correção do defeito e maior a probabilidade de correção adequada.

Fases do Teste de Software

- Teste de Unidade;
- Teste de Integração, e
- Teste de Sistema.

Etapas do Teste de Software

- Planejamento;
- Projeto de Casos de Teste;
- Execução do Programa com os Casos de Teste;
- Análise dos Resultados.

Caso de Teste

- Especificação de uma entrada para um *software* e a correspondente saída esperada.
 - Entrada — Conjunto de dados necessários para execução;
 - Saída esperada — Resultado da execução do programa.
- Bom caso de teste:
 - Aquele que tem alta probabilidade de revelar um erro.

Teste Funcional - Critérios

- Particionamento de Equivalência;
- Análise do Valor Limite;
- Grafo Causa-Efeito;
- *Error guessing*;
- Outros

Teste Funcional – Aplicação

- Identificar classes de equivalência;
 - Condições de entrada; e
 - Classes válidas e inválidas.
- Definir os casos de Teste:
 - Enumerar as classes de equivalência;
 - Casos de teste para as classes válidas; e
 - Casos de teste para as classes inválidas.

Teste Estrutural

- Caixa-branca (do inglês: *White Box*).
- Conhecendo os detalhes internos de um produto, testes são executados para verificar a execução de todas as estruturas que compõe os caminhos lógicos.
- Casos de teste visam percorrer todos os caminhos internos possíveis de um programa.
 - Verificam condições e/ou laços, e
 - Definições e uso das variáveis.

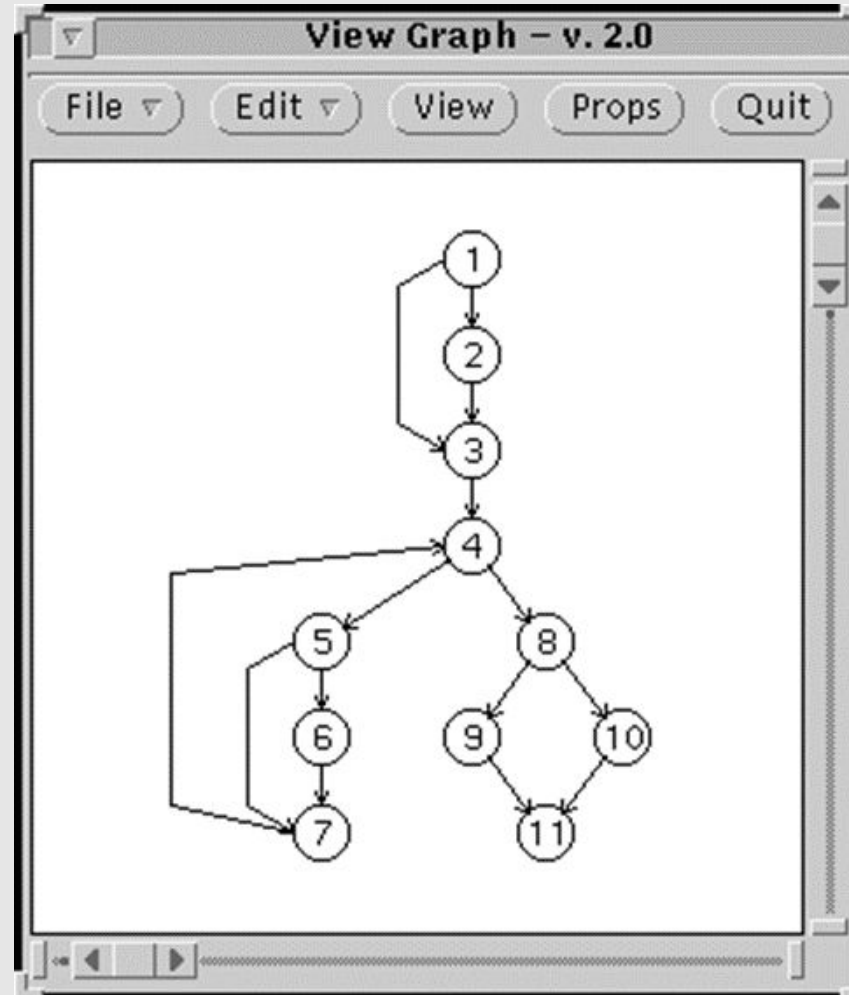
Teste Estrutural

- Maioria dos critérios utiliza uma representação de programa
 - Grafo de Fluxo de Controle (GFC) ou Grafo de Programa.
- Grafo é uma estrutura matemática composta por vértices (nós) e arcos (arestas).
 - Grafo de Programa $\rightarrow G = (N, E, s)$
 - N representa o conjunto de nós,
 - E conjunto de arcos,
 - s o nó de entrada.

Teste Estrutural – Grafo de Programa

- Nós → Blocos “indivisíveis”
 - Sem desvio para o meio do bloco;
 - Uma vez que o primeiro comando do bloco é executado, os demais comandos são executados sequencialmente.
- Arestas ou Arcos:
 - Representam o fluxo de controle entre os nós.

Teste Estrutural – Grafo de Programa



Fonte: Delamaro (2016)

Teste Estrutural – Critérios

- Baseados em Fluxo de Controle
 - Todos-nós, todas-arestas e todos-caminhos.
- Baseados em Fluxo de Dados
 - *Rapps e Weyuker*
 - Todas-defs, todos-usos, todos-p-usos, dentre outros.
 - Maldonado
 - Todos-potenciais-usos, todos-potenciais-usos/DU, outros.
- Baseados em Complexidade
 - Critério de McCabe.

Teste Baseado em Erros (ou Defeitos)

- Defeitos típicos são usados na definição dos requisitos de teste.
 - O programa a ser testado é alterado diversas vezes, criando um conjunto de programas alternativos (mutantes).
 - Defeitos são inseridos no programa original.
- Requisitos são derivados a partir dos erros frequentemente cometidos durante o processo de desenvolvimento.

Teste Baseado em Erros (ou Defeitos)

- Casos de teste devem mostrar diferença de comportamento entre o programa original e os programas mutantes.
- Critérios
 - Semeadura de Erros,
 - Teste de Mutação
 - Análise de Mutantes (unidade);
 - Mutação de Interface (integração).

Teste Baseado em Erros (ou Defeitos)

- Hipótese do Programador Competente

Um programa, produzido por um programador experiente, está correto ou está “próximo” do correto (desvios sintáticos).

- Efeito do Acoplamento

Casos de teste capazes de revelar erros simples, podem revelar erros mais complexos.

Teste Baseado em Erros (ou Defeitos)

- Aplicação do Teste de Mutação
 - Geração dos mutantes;



Fonte: Barbosa (2020)

Teste Baseado em Erros (ou Defeitos)

- Aplicação do Teste de Mutação
 - Execução do programa em teste;
 - Execução dos mutantes;
 - Mutante morto e mutante vivo;
 - Análise dos mutantes.
 - Mutante equivalente;
 - Inclusão de novos casos de teste;
 - Escore de mutação

Teste Baseado em Erros (ou Defeitos)

- Escore de mutação

$$ms(P,T) = \frac{DM(P,T)}{M(P) - EM(P)}$$

Fonte: Delamaro (2016)

- Onde:
 - $DM(P,T)$: número de mutantes mortos pelo conjunto de casos de teste T ;
 - $M(P)$: número total de mutantes gerados a partir do programa P ; e
 - $EM(P)$: número de mutantes gerados que são equivalentes a P .

Teste Baseado em Erros (ou Defeitos)

- Exemplo

```
1 void main()
2 {
3     int x, y, pow;
4     float z, ans;
5
6     scanf("%d %d", &x, &y);
7     if (y >= 0)
8         pow = y;
9     else
10        pow = -y;
11    z = 1;
12    while (pow-- > 0)
13        z = z*x;
14    if (y < 0)
15        z = 1 / z;
16    ans = z + 1;
17    printf("%-5.2f", ans);
18 }
```

Fonte: Delamaro (2016)

Teste Baseado em Erros (ou Defeitos)

- Exemplo - Operadores de mutação (Delamaro, 2016):
 - SSDL: eliminação de comandos;
 - ORRN: troca de operador relacional;
 - STRI: armadilha em condição de comando if; e
 - Vsrr: troca de variáveis escalares.

Teste Baseado em Erros (ou Defeitos)

- Exemplo
Operador ORRN

```
1 void main()
2 {
3     int x, y, pow;
4     float z, ans;
5
6     scanf("%d %d", &x, &y);
7     if (y >= 0)
8         pow = y;
9     else
10         pow = -y;
11     z = 1;
12     while (pow-- > 0)
13         z = z*x;
14     if (y < 0)
15         z = 1 / z;
16     ans = z + 1;
17     printf("%-5.2f", ans);
18 }
```

Fonte: Delamaro (2016)

Teste Baseado em Erros (ou Defeitos)

- Exemplo
Operador ORRN

12	if (y > 0)
13	if (y < 0)
14	if (y == 0)
15	if (y <= 0)
16	if (y != 0)
17	while (pow-- < 0)
18	while (pow-- == 0)
19	while (pow-- >= 0)
20	while (pow-- <= 0)
21	while (pow-- != 0)
22	if (y > 0)
23	if (y == 0)
24	if (y >= 0)
25	if (y <= 0)
26	if (y != 0)

Fonte: Delamaro (2016)

Teste Baseado em Erros (ou Defeitos)

- Exemplo - Mutante equivalente

Programa

```
1 void main()
2 {
3   int x, y, pow;
4   float z, ans;
5
6   scanf("%d %d", &x, &y);
7   if (y >= 0)
8     pow = y;
9   else
```

Mutante

12	if (y > 0)
13	if (y < 0)
14	if (y == 0)
15	if (y <= 0)
16	if (y != 0)

Fonte: Delamaro (2016)

Nesta aula...

- Entendemos a Atividade e as Fases do Teste de *Software*.
 - Reforçamos a importância dos requisitos.
 - Projeto de Casos de Teste.
- Conhecemos as Técnicas e Critérios de Teste de *Software*.
 - Funcional, Estrutural e Baseado em Erros (ou Defeitos).

Referências

BARBOSA, Ellen Francine et al. **Introdução ao teste de software**. João Pessoa: UFRJ/CEFET, 2000, Anais Simpósio Brasileiro de Engenharia de Software (SBES).

DELAMARO, Márcio Eduardo. **Introdução ao teste de software**. 2. Rio de Janeiro: GEN LTC, 2016. 1 recurso online. (SBC (Sociedade Brasileira de Computação)). ISBN 9788595155732. **Capítulo 1. Disponível na Biblioteca Digital da UFMS.**

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

